

Finding the Leak — Solution (OSINT sourcing of the dump)

English solution for the sourcing phase of the Data Breach lab. Analyst posture: threat intelligence / incident response. Goal: from nothing, work back to the `chat-coco.fr` dump circulating on the dark web, then verify its authenticity.

Two live artifacts are redacted from this public copy: the exact dark-web **post permalink** and the **direct download URL** of the dump. They resolve to the real personal data of thousands of non-consenting people; publishing a one-click path to that on a public repo would redistribute stolen data to the whole internet. They are provided to students **directly in class** (the dump itself is handed to you for the lab). Everything else — the exact method, the tools, the forum, the verification — is here.

Legal & ethical frame (read first)

Educational, **defensive** work. Same hygiene as a real analyst:

- **Dedicated / disposable VM**, connection over **Tor** (Tor Browser), no personal identity.
- **Observe and collect only**: authenticate to nothing, pay for nothing, download only the one artefact needed for analysis (the dump).
- **Forbidden**: re-sharing/reposting the data, cracking passwords, re-identifying or contacting victims.
- All exploitation (the analysis lab) is done **locally, read-only**.
- **Delete** the database and exports at the end of the exercise.

Holding and exploiting personal data from a leak **outside an authorised frame is illegal**. Here the frame is provided by the exercise.

The method: a chain

```
[.onion indexer]  -> [dark-web forum]  -> ["Data Leaks" board]  -> [leak post]
      OnionFind      Tenebris                               |
                                                           v
                                                           [file host]  -> [chatcoco.sql.gz]  ->
VERIFY
```

Keep provenance at every hop: URL, date/time of access (UTC), and the file's hash.

Step by step (reproducible)

1. Entry point — a hidden-service indexer

An **indexer** (dark-web search engine) finds `.onion` services by keyword without knowing the address in advance.

- Tool: **OnionFind** (`onionfind.com`).
- Keywords: the site name (`chat-coco`, `chatcoco`), or generic terms (`data leak`, `database dump`, `combolist`).
- Cross-check with other indexes/directories: **Ahmia** (`ahmia.fi`), **Tor.taxi**, **dark.fail** (a trusted onion directory — use it to avoid phishing clones).

2. A leak forum — Tenebris

The indexer leads to a community dark-web forum, **Tenebris**. Navigate its **communities / categories** and open the one that matters: "**Data Leaks**".

Reflex: verify the onion via a trusted directory (**dark.fail**) before visiting — well-known forums are massively cloned for phishing/scam.

3. The `chat-coco.fr` leak post

In the **Data Leaks** board, find the post dedicated to this leak. It describes the target site and the nature of the data, and links to the archive.

- Post permalink: **[redacted from the public repo — provided in class]**

4. The file host

The dump is usually not on the forum itself but on a **file-sharing service**.

- Download URL: **[redacted from the public repo — provided in class]**
- File obtained: `chatcoco.sql.gz` (~7.7 MB compressed, ~38 MB uncompressed).

5. Alternative path — the Wayback Machine

If the onion link is down (leaks are usually multi-source): `chat-coco.fr` was **defaced**; the hacked page (carrying the download link) may have been **archived**.

- **Wayback Machine** (`web.archive.org`) → query `chat-coco.fr` → find the defacement page and possibly the same download link.

Analyst lesson: a leak circulates in **several places** (forum, mirror, web archive, Telegram channels...). Cross-referencing **confirms authenticity** and **dates** its release.

Verify authenticity before touching it

Make sure the file is real and coherent (not a fake or a trap). Never execute it blindly.

```
# 1) File hash (keep it in the report = chain of custody)
sha256sum chatcoco.sql.gz
# 2) Real file type
file chatcoco.sql.gz          # -> gzip compressed data
# 3) Inspect the header WITHOUT full extraction
zcat chatcoco.sql.gz | head -20    # -- MariaDB dump ... Database: <name>
# 4) Count tables without importing (order-of-magnitude sanity check)
zcat chatcoco.sql.gz | grep -c '^CREATE TABLE'
```

Authenticity signals to expect:

- A real mariadb-dump export (structure + data).
- A database name consistent with the target site, with a shared-hosting (cPanel-style) prefix.
- A very high table count → signature of a social-network CMS (UNA/BoonEx family), consistent with a community site.

➡ Once authentic, proceed to the analysis lab (DataBreach_Lab.md).

What this phase contributes to the incident

- Confirms the leak and its **source** (site, technology, likely scale).
- Approximate **dating** of the release (forum post + web archive).
- A **verified artefact** (SHA-256) to base the impact analysis on.
- The basis to **notify** (in a real case: DPA within 72h, data subjects, complaint).

Marking

Item	Pts	What earns the points
Method & chain	30	indexer → forum → board → post → host, reproducible
Right tools	20	OnionFind/Ahmia/dark.fail used; onion verified before visiting
Cross-source / dating	20	Wayback (or a second source) used to confirm & date
Authenticity verification	20	hash, file, header, table count

Item	Pts	What earns the points
OPSEC & legal	10	Tor + VM, observe-only, provenance, legal frame respected